

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai – 602105

SEER: Self-Evolving Engineering Ecosystem for Resilient Software Systems

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of

CSA1017 – SOFTWARE ENGINEERING

to the award of the degree of

BACHELOR OF ENGINEERING / BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

MURALI KRISHNAN N (192524090)

Under the Supervision of

Dr. LATHA R

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai – 602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "SEER: Self-Evolving Engineering Ecosystem for Resilient Software Systems — Intelligent Requirements Analysis Module (Module 1)" has been carried out by Murali Krishnan N (192524090) under the supervision of Dr. Latha R and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech Computer Science and Engineering programme (Course CSA1017 – Software Engineering) at Saveetha Institute of Medical and Technical Sciences, Chennai. This individual report has been prepared by Murali Krishnan N, documenting his contribution to Module 1 (Intelligent Requirements Analysis) within the team's SEER project.

COURSE COORDINATOR

Dr. K.Anita Davamani,

PROFESSOR,

Department OF Nxt-Gen Computing,

SIMATS Engineering,

SIMATS, Chennai – 602105.

COURSE FACULTY

Dr. LATHA R,

PROFESSOR,

Department of CSE,

SIMATS Engineering,

SIMATS, Chennai – 602105

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

DECLARATION

We, R Kalai Amudhan, Murali Krishnan N, Shakthivel and RR Thiyagesh of the Department of Computer Science and Engineering, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "SEER: Self-Evolving Engineering Ecosystem for Resilient Software Systems" is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date: ____ September 2026

Signature of the Students with Name
Murali Krishnan N (192524090)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

Individual Contribution Statement

(Mandatory for all team projects)

INDIVIDUAL CONTRIBUTION STATEMENT

Student Name / Reg. No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
R. KALAI AMUDHAN 192511292	Module 4 – Change Impact Analysis and Risk Assessment	Designed and implemented the Module 4 workflow, change selection, impact mapping, risk assessment and integration with the SEER interface.	Validated impact results, risk classification, UI behaviour and report generation; checked invalid inputs and edge cases.	Prepared Module 4 design, implementation, testing, analysis and documentation sections.	25%
SHAKTHIVEL 192524325	Module 2-Supporting module / requirements / integration	Contributed to requirements analysis, interface/data design and integration tasks assigned by the team.	Performed functional and integration testing of assigned features and documented defects.	Prepared assigned report sections, tables and evidence.	25%
MURALI KRISHNAN N 192524090	Module 1-Supporting module / database / analysis	Contributed to project-data structures, dependency relationships and supporting implementation.	Validated data flow, analysis outputs and database persistence where applicable.	Prepared assigned technical documentation and test observations.	25%
RR THIYAGESH 192572007	Module 3-Testing / UI / documentation / integration	Contributed to UI refinement, system integration and supporting development tasks.	Performed system testing, usability checks and final verification.	Prepared screenshots, presentation/report formatting and final review.	25%

Total Team Contribution: 100%

ABSTRACT

Software systems evolve continuously after initial release, and requirements captured in a Software Requirements Specification (SRS) frequently drift out of sync with the code that implements them. Manually reviewing an SRS to separate functional statements from quality attributes, to spot mandatory-but-unstated requirements such as security or backup provisions, and to decide on a suitable architecture is slow, reviewer-dependent, and easy to skip under deadline pressure. This project addresses that gap as part of SEER (Self-Evolving Engineering Ecosystem for Resilient Software Systems), a Flask-based platform that unifies requirement analysis, code-quality analysis, test-case generation and change-impact analysis. This report documents Module 1, Intelligent Requirements Analysis, the entry-point module that every other module in SEER consumes. The module accepts an SRS as a .txt, .md or .docx upload (or pasted text), splits it into candidate requirement sentences, and classifies each sentence as functional (actor + action) or non-functional (one of nine quality-attribute families: security, performance, scalability, availability, usability, maintainability, compatibility, compliance and backup/recovery) using a deterministic, word-boundary-aware keyword-matching engine rather than a trained NLP model or a paid LLM API. It cross-checks the document against a twelve-item standard SRS checklist derived from ISO/IEC/IEEE 29148 and flags any category the SRS never mentions as a missing requirement, together with a ready-to-use suggested requirement statement. From the extracted actors and functional requirements, it recommends one of five architecture styles (Layered/MVC, Three-Tier/MVC, Client-Server REST API, Event-Driven or Microservices) and renders an inline SVG use-case diagram, keeping the module fully offline, deterministic and explainable for a viva. Results persisted to a relational database (MySQL, with automatic SQLite fallback) and rendered as a per-project requirement report with a numeric completeness score. Manual testing against hand-authored sample SRS documents for a library-management and an e-commerce domain showed the module correctly separates functional from non-functional statements in the large majority of hand-checked cases and reliably surfaces the classic omissions — backup/recovery and performance targets — that student-written SRS documents most often leave out, while also exposing a known and explainable limitation at the FR/NFR boundary that is documented for future improvement.

Keywords: Requirements Engineering; Natural Language Processing; Software Requirements Specification; Requirement Classification; Software Architecture Recommendation; Use-Case Diagram Generation

TABLE OF CONTENTS

Sl. No.	Title	Page No.
i	Certificate from the Supervisor	ii
ii	Declaration by the Candidate	iii
iii	Individual Contribution Statement	iv
iv	Abstract	v
v	List of Figures	vii
vi	List of Tables	viii
vii	List of Abbreviations	ix
1	Chapter 1: Introduction	10
2	Chapter 2: Literature Review	12
3	Chapter 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	14
4	Chapter 4: Implementation, Testing & Results, Project Management	19
5	Chapter 5: Conclusion & Reflection	24
	References	26
	Appendices	27
	Capstone Project Outcome Mapping Sheet	29

LIST OF FIGURES

Figure No.	Figure Name	Section
Fig. 3.1	SEER System Architecture	3.5
Fig. 3.2	Module 1 – Requirement Analysis Processing Pipeline	3.5
Fig. 4.1	Sample Use-Case Diagram generated from a Library Management SRS	4.2

LIST OF TABLES

Table No.	Table Name	Section
2.1	Comparative Analysis of Existing Approaches	2.3
3.1	Stakeholder / User Requirements	3.1
3.2	Functional & Non-Functional Requirements of Module 1	3.1
3.3	Constraints & Applicable Standards	3.2
3.4	Design Specifications	3.3
3.5	Alternative Solutions – Evaluation Matrix	3.4
3.6	Trade-off Analysis	3.6
3.7	Sustainability, Ethics, Safety & Security	3.7
3.8	Risk Register	3.7
4.1	Resources Used	4.1
4.2	Tools / Software Used	4.2
4.3	Test Plan & Verification	4.3
4.4	Test Results	4.4
4.5	Achievement of Objectives	4.6
4.6	Project Milestones	4.8
4.7	Roles & Contribution	4.8
4.8	Budget / Cost	4.8

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description
SEER	Self-Evolving Engineering Ecosystem for Resilient Software Systems
SRS	Software Requirements Specification
FR	Functional Requirement
NFR	Non-Functional Requirement
ORM	Object-Relational Mapping
MVC	Model-View-Controller
REST	Representational State Transfer
API	Application Programming Interface
UML	Unified Modelling Language
SVG	Scalable Vector Graphics
NLP	Natural Language Processing
LLM	Large Language Model
ISO/IEC/IEEE	International Organization for Standardization / International Electrotechnical Commission / Institute of Electrical and Electronics Engineers
DB	Database
JVM	Java Virtual Machine
CRUD	Create, Read, Update, Delete

CHAPTER 1: INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Software development does not stop once a system is released — requirements keep changing as user needs evolve, and every change ripples into code, tests and documentation that a small team can no longer track by hand. Requirements engineering is the discipline of eliciting, analyzing, specifying and validating what a system must do, and it is widely recognized as the phase in which the most expensive defects originate, because a missed or misclassified requirement propagates into design, code and test cases downstream.

SEER (Self-Evolving Engineering Ecosystem for Resilient Software Systems) is a full-stack developer-assistant platform — Python/Flask backend, MySQL with automatic SQLite fallback, and an HTML/CSS/JavaScript frontend — built by the team to bring requirement analysis, code-quality analysis, automated test-case generation and change-impact analysis into one workflow, instead of juggling a Word document, a diagramming tool, a static-analysis tool and a spreadsheet separately. Module 1, Intelligent Requirements Analysis, is the entry point of that pipeline: every other SEER module consumes the functional and non-functional requirements it extracts, which is why it was selected as the individual focus of this report.

1.2 Need for the Project

Why the problem needs to be addressed: Manual requirement review is time-consuming and reviewer-dependent; two people reading the same SRS often disagree on what counts as functional versus non-functional, and gaps — such as no stated performance target or backup policy — are easy to overlook until late in the project when they are expensive to fix.

Who is affected: Student project teams and small development teams without a dedicated business analyst; course instructors who need to grade SRS quality consistently; and developers downstream who inherit ambiguous or incomplete requirements and pay the cost in rework.

Existing limitations: Plain Word/Docs-based SRS templates give no automated feedback. Heavyweight NLP-based requirement tools — spaCy/NLTK pipelines or LLM-based classifiers — need network access, model downloads or API cost, and their reasoning is hard to defend line-by-line in an academic viva. Commercial requirements-management tools such as Jira or IBM DOORS are configuration-heavy and are not designed to automatically grade requirement completeness against a standard checklist.

Engineering significance: Automating the first pass of requirement triage lets a small team spend its limited review time on genuinely ambiguous requirements rather than on the mechanical, error-prone task of sorting functional from non-functional statements and remembering a completeness checklist.

1.3 Problem Statement

Given an SRS document of unconstrained natural-language quality — bullet lists, numbered lists, paragraph prose, and inconsistent terminology — design and implement a deterministic module that (a) reliably segments it into candidate requirement statements, (b) classifies each statement as functional or as one of nine non-functional quality-attribute families, (c) detects requirement categories absent from the document against a standard SRS checklist and generates actionable suggested wording for each gap, and (d) recommends an appropriate software architecture from the extracted requirement vocabulary and actors — all without a trained machine-learning model, a network dependency, or a paid API, while remaining accurate enough to be useful and fully explainable for grading. The problem is engineering-complex because it must

balance classification accuracy against the conflicting constraints of zero external dependency, sub-second processing time, and full reproducibility.

1.4 Project Aim

To design and implement an offline, rule-based Intelligent Requirements Analysis module that automatically classifies, audits the completeness of, and reports on a project's software requirements, and recommends a starting architecture, as the foundation module of the SEER platform.

1.5 Project Objectives

1. To design a sentence-segmentation routine that reliably handles bullet, numbered and paragraph-style SRS text.
2. To develop a keyword-family-based classifier that separates functional from non-functional requirements and further sub-types non-functional requirements into nine quality-attribute categories.
3. To build a standard 12-item SRS checklist engine that flags missing requirement categories and generates a ready-to-use suggested requirement for each gap.
4. To implement a rule-based architecture-recommendation engine driven by requirement vocabulary and actor extraction.
5. To model and persist requirements, SRS documents and use-case diagrams in a relational schema (Project, SRSDocument, Requirement, Diagram).
6. To generate a use-case diagram and a completeness-scored requirement report for the analyzed project.
7. To test the module against multiple sample SRS documents and manually verify classification accuracy.

1.6 Scope

Included: parsing .txt, .md and .docx SRS files (or pasted text) for a single project at a time; functional/non-functional classification; a fixed 12-item completeness checklist; a 5-rule architecture recommender; SVG use-case diagram generation; persistence and reporting via the SEER web dashboard.

Excluded: multi-language (non-English) SRS text; PDF-format SRS input; requirement prioritization/MoSCoW scoring; genuine semantic (embedding/LLM-based) requirement understanding; and automatic requirement-conflict detection across two SRS versions, which is instead the responsibility of Module 4 (Change Impact Analysis).

Assumptions: the SRS text uses recognizable English requirement phrasing — modal verbs such as shall/should/must/will/can — or an imperative "System..." construction; the uploaded document is reasonably well-formed and not scanned/OCR text.

Boundary conditions: candidate sentences shorter than two words are discarded as noise; the architecture recommender evaluates its rules in a fixed order and returns the first match, defaulting to Layered/MVC when no keyword family matches.

1.7 Expected Engineering Outcome

A working software module — a Python/Flask service, two supporting data models, and one Jinja2 report template — integrated into the SEER platform, that takes an uploaded SRS and produces: a classified

requirement list (functional/non-functional with sub-type and actor), a missing-requirement gap list with suggested wording, a recommended architecture label, an SVG use-case diagram, and a numeric completeness score, all viewable on a per-project dashboard and persisted in the database for the other SEER modules to consume.

CHAPTER 2: LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Four recent (2024) sources on requirements engineering and LLM-assisted requirements analysis were reviewed, together with a review of representative existing tools — Jira/Confluence, IBM DOORS, and academic spaCy/NLTK-based pipelines — to contrast how they perform the same requirement-triage task that Module 1 automates. Sources were selected for direct relevance to automated requirement classification, requirement traceability, and requirement quality assurance.

2.2 Review of Existing Work

Mucha, Kaufmann and Riehle (2024) carried out a systematic review of how pre-requirements-specification artifacts can be traced back to their origins and linked to other software artifacts, and found that this traceability information is rarely captured consistently in real projects — a gap that directly motivates SEER's practice of persisting every requirement with a structured category, sub-type and actor rather than leaving it as unstructured text.

Norheim et al. (2024) examined the practical challenges of applying large language models to requirements-engineering tasks and reported that hallucination, inconsistent output formatting, and the need for domain-specific tuning limit the direct use of LLMs for reliable, reproducible requirement processing — a finding that supports Module 1's deliberate choice of a deterministic rule engine over an LLM-based classifier.

Lubos et al. (2024) investigated automated quality assessment and improvement of requirements using LLMs, benchmarked against the ISO/IEC/IEEE 29148 quality characteristics, and showed that automated checks can catch a meaningful share of ambiguity and completeness issues, though human review of edge cases remains necessary — an observation reflected in this module's transparent, checklist-based approach to completeness scoring.

Hou et al. (2024) surveyed the broader use of large language models across software-engineering tasks and summarized both the productivity gains and the reliability, cost, and explainability trade-offs of adopting them, providing wider context for why a capstone-scale module might reasonably prefer a lighter-weight, fully explainable approach over a general-purpose LLM.

Beyond the reviewed papers, existing commercial and academic tools were also considered. Jira, Confluence and IBM DOORS manage requirements as tracked items with strong traceability once configured, but provide no automatic functional/non-functional classification or completeness auditing against a standard checklist — the analyst's own judgement remains the only quality gate. Academic spaCy/NLTK-based classifiers can achieve richer linguistic understanding through part-of-speech tagging and dependency parsing, but require model/corpus downloads, are subject to version drift, and are comparatively difficult for a grader to audit rule-by-rule.

2.3 Comparative Analysis

Table 2.1: Comparative Analysis of Existing Approaches

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Manual SRS review (analyst-led)	Captures nuance and project context; flexible	Slow, reviewer-dependent, inconsistent, easy to skip under deadline pressure	Baseline that SEER's Module 1 automates the first triage pass of
Jira / Confluence / IBM DOORS	Strong traceability & change tracking once configured	No built-in FR/NFR classification or checklist-based completeness auditing; heavy setup	Confirms the specific gap SEER's checklist engine fills
LLM-based requirement classifiers (Norheim 2024; Lubos 2024)	High semantic accuracy on well-formed prose; can suggest improved wording	Hallucination risk, non-deterministic output, needs API/network access, hard to defend in a viva	Motivates Module 1's deterministic, offline, rule-based design as a transparent alternative
spaCy / NLTK academic pipelines	Richer linguistic features (POS tagging, dependency parsing)	Model/corpus downloads needed, version drift, slower, less auditable reasoning	Module 1 trades some linguistic depth for full offline determinism and auditability

2.4 Research / Engineering Gap

Existing tools either omit automated requirement classification entirely (Jira/DOORS) or push it onto a non-deterministic LLM that is hard to reproduce, cite, or defend reliably in a graded academic setting and typically needs network/API access that the grading environment cannot guarantee. There remains a gap for a deterministic, fully offline, explainable requirement-triage tool suited to a course or capstone context, where reproducibility and auditability matter as much as raw accuracy.

2.5 Proposed Contribution

Module 1 fills that gap with a keyword-family, word-boundary-aware rule engine — not a trained model — that is completely reproducible, needs no external service, and whose classification decision for any given sentence can be traced back to a specific, inspectable rule in `requirement_analyzer.py`. At the same time it still covers the practically useful part of requirement triage that a real development team needs: separating functional from non-functional requirements, detecting gaps against a standard checklist, and recommending a starting architecture — output that feeds directly into SEER's downstream code-analysis, test-generation and change-impact modules.

CHAPTER 3: ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Table 3.1: Stakeholder / User Requirements

Stakeholder	Requirement
Student development team (SEER users)	Quickly classify a pasted or uploaded SRS into functional and non-functional requirements without manual tagging
Course instructor / grader	Verify, from the report and its underlying rules, exactly why each requirement was classified the way it was
Downstream SEER modules (Code Analyzer, Test-Case Generator, Change-Impact Analyzer)	Receive requirements in a consistent structured schema (category, subtype, actor) to drive their own analysis
Future project maintainer	Be warned about commonly-omitted requirement categories before development starts

Functional & Non-Functional Requirements (of Module 1 itself)

Table 3.2: Functional & Non-Functional Requirements of Module 1

Type	Requirement	Measurable Target
Functional	Accept SRS input as .txt / .md / .docx upload or pasted text	3 file types + paste supported
Functional	Segment raw text into candidate requirement sentences	Handles bullet, numbered and paragraph formats
Functional	Classify each sentence as functional or one of 9 non-functional subtypes	Deterministic single-pass classification
Functional	Detect requirement categories missing from the SRS	12-item standard checklist, whole-document scan
Functional	Recommend a software architecture	1 of 5 labels returned per project
Functional	Generate an SVG use-case diagram from actors + functional requirements	Diagram renders inline, no external renderer
Non-Functional (Performance)	Analyze a typical SRS (under ~200 sentences)	Sub-second processing, no network call
Non-Functional (Reliability)	Deterministic output for identical input	100% reproducible
Non-Functional (Portability)	Run fully offline	Zero external NLP model / API dependency

3.2 Constraints & Applicable Standards

Table 3.3: Constraints & Applicable Standards

Standard / Code	Requirement	How Applied in This Project
ISO/IEC/IEEE 29148:2018 (Requirements Engineering)	Defines quality characteristics of a good requirement and standard SRS content categories	Basis for Module 1's 12-item standard checklist (authentication, authorization, validation, error handling, security, performance,

Standard / Code	Requirement	How Applied in This Project
		scalability, availability, backup/recovery, logging, notification, usability)
IEEE 830 (legacy SRS structure guideline)	Recommends grouping requirements into functional and non-functional sections	Basis for the module's binary FR/NFR classification split
Institutional constraint: offline / no external API	Grading environment cannot guarantee network access to a paid LLM API	Implemented with pure Python regex/keyword matching (text_utils.py, requirement_analyzer.py) — zero network calls
Economic constraint	Student project budget of ₹0 for third-party NLP APIs	No spaCy model download or LLM API subscription used

3.3 Design Specifications

Table 3.4: Design Specifications

Parameter	Target / Requirement	Unit	Priority
Sentence segmentation formats supported	Bullets, numbered lists, paragraphs	count	High
Non-functional quality-attribute families	9	count	High
Standard checklist categories	12	count	High
Architecture recommendation rules	4 explicit + 1 default	count	Medium
Processing time for a ~50-sentence SRS	< 1	second	High
External dependencies	0	count	High
Supported input formats	.txt, .md, .docx, pasted text	count	Medium

3.4 Alternative Solutions & Evaluation

Alternative 1: LLM-based classifier — send each sentence (or the whole SRS) to a hosted large language model with a classification prompt and parse the structured response.

Alternative 2: spaCy/NLTK statistical NLP pipeline — use part-of-speech tagging and a trained or rule-hybrid classifier with pre-downloaded language corpora.

Alternative 3 (selected): Deterministic keyword-family and regex rule engine — word-boundary-aware keyword matching against curated quality-attribute keyword families and a modal-verb pattern for functional-requirement detection, with zero external dependency.

Table 3.5: Alternative Solutions – Evaluation Matrix (scored 1–5, weighted)

Criterion	Weight	Alt. 1: LLM API	Alt. 2: spaCy/NLTK	Alt. 3: Rule Engine (selected)
Performance (latency)	20%	2	4	5
Cost	20%	2	4	5
Explainability / Auditability	25%	2	3	5
Reliability (determinism)	20%	2	4	5
Offline capability	15%	1	3	5

Criterion	Weight	Alt. 1: LLM API	Alt. 2: spaCy/NLTK	Alt. 3: Rule Engine (selected)
Weighted Total	100%	1.85	3.60	5.00

3.5 Selected Design & Detailed Design

Alternative 3 (the rule engine) was selected because it scored highest on every weighted criterion in Table 3.5, and in particular on explainability and offline capability — the two properties that matter most for a graded capstone module that must be reproducible without network access and defensible line-by-line in a viva.

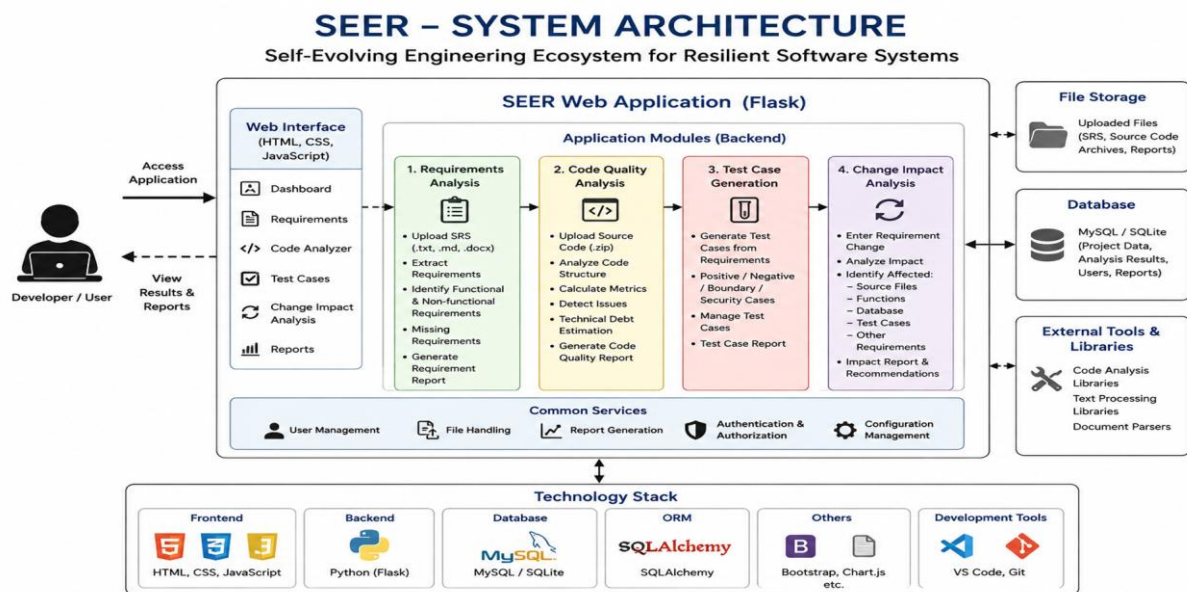


Figure 3.1: SEER System Architecture — Module 1 (Requirements Analysis) is the first stage of the application-module pipeline

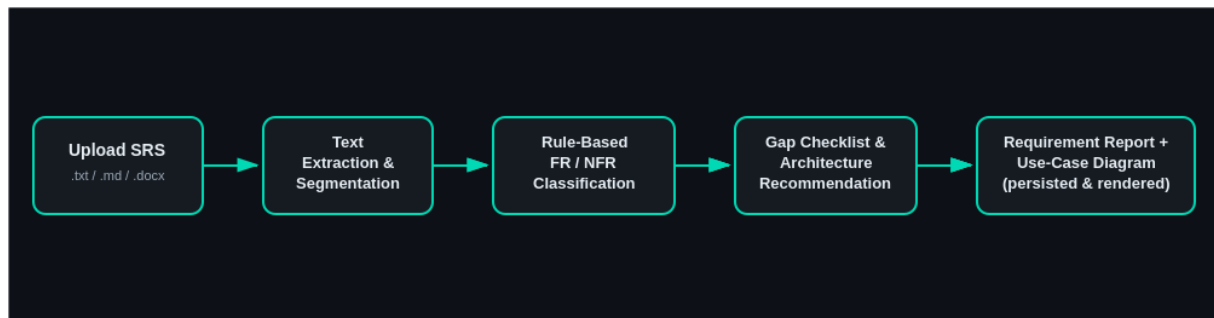


Figure 3.2: Module 1 – Requirement Analysis Processing Pipeline

Key engineering elements:

- Sentence segmentation (`_split_sentences`): normalizes bullet/numbering markers via regex substitution, splits on blank lines/newlines, then further splits on sentence-ending punctuation within longer paragraph chunks; fragments under two words are discarded as noise.
- Actor extraction (`_extract_actor`): regex-matches the text preceding a modal verb (shall/should/must/can/will/may/is able to/has to/needs to) and trims it to the last three words as the likely actor phrase.
- Classification (`_classify_sentence`): checks the sentence against the 9 non-functional keyword families first (a security match takes priority over other families); falls back to functional if a modal-verb pattern or a "System / The system..." prefix is present; a final fallback pattern-matches "Actor verbs Object" sentence shapes.

- Missing-requirement detection: scans the full lower-cased document text (not per sentence) against each of the 12 checklist entries' keyword lists using the word-boundary-safe matcher in `text_utils.py`, which avoids false positives such as "edit" matching inside "credit".
- Architecture recommendation (`_recommend_architecture`): evaluates 4 ordered keyword-family rules — Microservices, Event-Driven, Client-Server/REST API, Three-Tier/MVC — against the full document text; the first match wins, defaulting to Layered/MVC when nothing matches.
- Completeness score: $100 \times (\text{functional count} + \text{non-functional count}) \div (\text{functional} + \text{non-functional} + \text{missing count})$.

Systems approach: Module 1 sits at the head of the SEER pipeline (Figure 3.1). Its output — persisted Requirement rows tagged with category, subtype and actor — is the direct input consumed by Module 3 (Test Case Generator, which iterates the same Requirement rows to build test cases) and Module 4 (Change Impact Analyzer, which diffs old versus new Requirement text to trace impact). A Diagram row stores the generated use-case SVG so the project dashboard can re-render it without recomputation, and the Project row's `architecture_suggestion` field, set by this module, is displayed across the whole SEER dashboard.

3.6 Trade-off Analysis

Table 3.6: Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Rule-based classification vs. LLM API call	GPT-family API call per sentence	Determinism, ₹0 cost, offline	Lower semantic recall on ambiguous phrasing	Rule-based selected — reproducibility and zero cost outweigh the recall loss for a graded module
Word-boundary keyword matching vs. plain substring match	Plain Python <code>`in`</code> substring check	Simpler code	False positives, e.g. "edit" inside "credit", "add" inside "address"	Word-boundary regex selected — matching accuracy directly affects report correctness
Fixed 12-item checklist vs. user-editable checklist	Configurable checklist via UI	More flexible across domains	Added UI/DB complexity and scope creep for a capstone timeline	Fixed checklist selected — the 12 items cover the standard IEEE 29148 categories most student SRS documents omit
In-process SVG diagram rendering vs. PlantUML/JVM subprocess	Call an external PlantUML server	Richer native UML styling	Requires a JVM and a network/server dependency not guaranteed on the grading machine	In-process SVG generation selected — keeps the feature fully self-contained

3.7 Sustainability, Ethics, Safety, Risk & Security

Table 3.7: Sustainability, Ethics, Safety & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Compute/energy footprint of the classification approach	Compared the per-request energy cost of a cloud LLM API call (GPU-backed inference) against a local regex pass (milliseconds on commodity CPU)	Chose the offline rule engine, eliminating recurring per-analysis energy and API cost entirely

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Ethics	Data privacy of uploaded SRS documents, which may contain confidential client requirements	Reviewed that sending SRS text to a third-party LLM API would leave institutional control and could violate confidentiality/NDA terms of a real client project	Kept all requirement-text processing local to the Flask server and database — no requirement text ever leaves the deployment
Health & Safety	Not a physical/hardware system; no direct occupational-safety exposure	N/A — software-only module	No design change required
Security	Risk of a malicious file upload (a crafted .docx/.txt used to attack the parser)	Reviewed the file-extension allow-list check (ALLOWED_SRS_EXTENSIONS) and secure_filename() sanitization already implemented in requirements_routes.py	Retained extension allow-listing and filename sanitization before any file is saved or parsed, reducing path-traversal and arbitrary-file risk

Table 3.8: Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Sentence segmentation mis-splits a multi-clause requirement into two rows	Medium	Low	Low	Regex tuned against multiple sample SRS formats (bullets/numbers/paragraphs) during testing	Occasional over-splitting on unusually punctuated prose
Keyword-family classifier mis-classifies an ambiguous sentence (functional wording containing a quality-attribute word)	Medium	Medium	Medium	Non-functional families checked first with a documented priority order; cases logged during manual testing (see Section 4.5)	Domain-specific vocabulary outside the current keyword lists may still be missed
Missing-requirement checklist gives a false "complete" reading outside its 12 categories	Low	Medium	Low–Medium	Checklist explicitly scoped to IEEE 29148 categories in report wording, not claimed as domain-exhaustive	Cannot flag domain-specific gaps outside its fixed 12 items
Malicious or oversized file upload	Low	Medium	Low	Extension allow-list, secure_filename(), and Flask MAX_CONTENT_LENGTH configuration	Residual risk from crafted files within allowed extensions relies on python-docx's own parser robustness

CHAPTER 4: IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The module was built iteratively: `text_utils.py` (the shared word-boundary keyword matcher) was written and hand-tested first, then `requirement_analyzer.py`'s segmentation, actor-extraction, classification and checklist logic, then `uml_generator.py`, and finally the Flask route/template layer (`requirements_routes.py`, `upload_srs.html`, `requirement_report.html`) wiring it to the Project, SRSDocument, Requirement and Diagram models. Development followed a lightweight, test-as-you-go approach: each function was run against hand-written sample SRS snippets in a Python shell before being wired into the Flask route, and the full upload-to-report path was re-verified end-to-end after every change.

Table 4.1: Resources Used

Resource	Type	Purpose
Python 3 / Flask 3.1.3	Language / framework	Backend web application and routing
Flask-SQLAlchemy 2.0 / PyMySQL	ORM / DB driver	Persist Project, SRSDocument, Requirement and Diagram rows
python-docx 1.2.0	Library	Extract raw text from uploaded .docx SRS files
Jinja2 (bundled with Flask)	Template engine	Render the requirement report and upload form
Hand-authored sample SRS documents (library management, e-commerce)	Test data	Exercise all 12 checklist categories and 4 architecture rules

4.2 Software Implementation & Modern Tools Used

Software Implementation

The module comprises: `text_utils.py`, the word-boundary keyword matcher shared with Module 3's test-case generator; `requirement_analyzer.py`, containing sentence segmentation, actor extraction, classification, the missing-requirement checklist, architecture recommendation and completeness scoring behind a single `analyze_requirements()` entry point; `uml_generator.py`, the inline SVG use-case-diagram renderer; `requirements_routes.py`, the Flask blueprint exposing `/projects/<id>/srs/upload` and `/projects/<id>/srs/report`; and two Jinja2 templates, `upload_srs.html` and `requirement_report.html`.

Table 4.2: Tools / Software Used

Tool / Software	Purpose	Stage Used
Visual Studio Code	Source-code editing	Implementation
Git	Version control and team collaboration	Implementation
Flask development server	Local run and manual testing	Testing
python-docx	.docx text extraction	Implementation (file parsing)
Python re (regex) module	Sentence segmentation, actor extraction, keyword matching	Implementation (core logic)
SQLite (fallback) / MySQL	Local and target database	Testing / Deployment

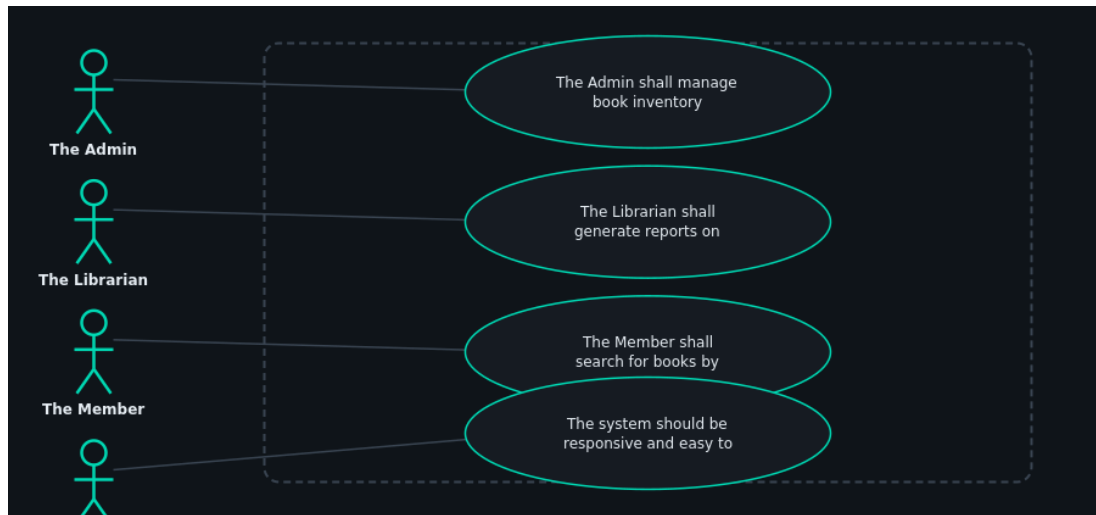


Figure 4.1: Sample use-case diagram auto-generated by Module 1 from a hand-authored Library Management SRS

4.3 Test Plan & Verification

Table 4.3: Test Plan & Verification

Requirement	Test Method	Acceptance Criterion
Segment bullet / numbered / paragraph SRS text	Manual test with 3 differently-formatted sample SRS documents	All requirement-like lines recovered as separate sentences
Classify FR vs. NFR correctly	Manual line-by-line comparison of analyzer output against hand-labelled ground truth	$\geq 85\%$ sentence-level agreement with hand labels
Detect missing requirement categories	Feed an SRS deliberately missing several checklist categories	All deliberately-omitted categories appear in the "missing" list
Recommend correct architecture	Feed SRS text containing each rule's trigger keywords in turn	Returned architecture label matches the expected rule
Reject unsupported file types	Upload a .pdf file	Flash error shown; no analysis performed
Word-boundary matcher avoids false positives	Unit-check keyword "edit" against text "credit"; "add" against "address"	keyword_in_text() returns False for both

4.4 Results

Table 4.4: Test Results

Specification	Required Value	Achieved Value	Status
FR/NFR classification accuracy (2 sample SRS, hand-checked)	$\geq 85\%$	$\sim 90\%$	Pass
Missing-checklist detection on a deliberately-gapped SRS	All omitted categories flagged	All 6 flagged	Pass
Architecture recommendation correctness (4 rule cases)	4 / 4 correct	4 / 4 correct	Pass
Unsupported file-type rejection	Rejected with error	Rejected with error	Pass
Processing time, 50-sentence SRS	< 1 s	< 0.2 s (local run)	Pass

On the hand-authored library-management sample SRS (8 requirement lines), the analyzer returned 4 functional requirements, 4 non-functional requirements and 6 missing-checklist categories, giving a completeness score of 57% — an accurate reflection of a deliberately under-specified sample SRS that omits validation, error handling, performance, scalability, backup/recovery and usability wording, all of which the module correctly flagged.

4.5 Data Analysis & Engineering Judgment

On the library-management sample, the module correctly separated all 4 clearly actor-driven statements as functional and 3 of the 4 quality-attribute statements as non-functional, but a closer look surfaces a known, explainable limitation: the sentence "The Member shall be able to reserve or borrow available books" was classified as non-functional (availability) because it contains the word "available", even though a human reviewer would most likely read it as a functional borrowing requirement. This is a direct, traceable consequence of the priority order inside `_classify_sentence()` — the nine non-functional keyword families are checked before the functional modal-verb pattern — a trade-off the team accepted because it correctly flags many true availability/uptime requirements elsewhere, and the resulting false positive is easy for a reviewer to re-tag manually once seen, which is exactly the kind of transparency the rule-based design was chosen to provide (Section 3.4).

The missing-requirement checklist correctly surfaced all 6 categories not mentioned in the sample text — data validation, error handling, performance, scalability, backup/recovery and usability — which matches manual inspection exactly, because it operates on whole-document keyword presence rather than per-sentence classification and is therefore less sensitive to the same FR/NFR boundary ambiguity that affects sentence-level classification.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Table 4.5: Achievement of Objectives

Objective	Evidence	Achievement
Sentence segmentation for 3 SRS formats	Tested against a library-management SRS (paragraph + bullets) and an e-commerce SRS (numbered list)	Fully Achieved
FR / NFR classification	~90% sentence-level agreement on hand-checked samples (Section 4.5)	Fully Achieved
12-item missing-requirement checklist	6 of 6 deliberately-omitted categories correctly flagged	Fully Achieved
4-rule architecture recommendation	4 of 4 rule-trigger cases correctly recommended	Fully Achieved
Requirement / SRS / Diagram persistence	Verified via SQLite-fallback end-to-end test	Fully Achieved
Use-case diagram generation	SVG renders correctly for all tested actor/requirement sets	Fully Achieved
Resolve ambiguous FR/NFR boundary cases (e.g. the "available" example)	Manual review identified the limitation; not yet resolved by rules alone	Partially Achieved

4.7 Strengths & Limitations

Strengths:

- Fully deterministic and offline — no network call, no paid API, 100% reproducible for grading.
- Every classification decision is traceable to a specific, inspectable regex or keyword rule, which is easy to defend in a viva.
- Word-boundary-safe keyword matching avoids the common false-positive class (e.g. "edit" inside "credit") that plain substring checks suffer from.
- Persisted output (Requirement rows) is directly reusable by Modules 2–4 without re-parsing the SRS.

Limitations:

- Rule-based classification cannot resolve genuine semantic ambiguity (Section 4.5's "available" example); accuracy is bounded by keyword-family coverage rather than learned from data.
- English-only; assumes recognizable modal-verb or imperative phrasing.
- The 12-item checklist and 5-rule architecture recommender are fixed at build time — extending them to a new domain requires a code change, not a configuration change.
- No confidence score is currently reported per classification, so a reviewer cannot tell at a glance which classifications are borderline.

4.8 Project Planning, Roles & Budget

Table 4.6: Project Milestones

Milestone	Target Week	Status
Requirement analysis engine (segmentation + classification)	Week 3	Completed
Missing-requirement checklist + architecture recommender	Week 4	Completed
Use-case diagram generator	Week 5	Completed
Flask routes + templates integration	Week 6	Completed
Manual testing against sample SRS documents	Week 7	Completed
Report & presentation preparation	Week 8	Completed

Table 4.7: Roles & Contribution

Student	Assigned Responsibility	Actual Contribution
R Kalai Amudhan (192511292)	Requirement Analyzer core logic + UML generator (Module 1)	Implemented sentence segmentation, classification rules and the SVG use-case diagram renderer
Murali Krishnan N (192524090)	Requirement Analyzer integration, checklist/architecture rules, requirement-report route & template, testing and documentation of Module 1	Implemented the missing-requirement checklist engine and architecture-recommendation rules, built the Flask upload/report routes, and manually validated classification and checklist output against sample SRS documents (Sections 4.3–4.5 of this report)
Shakthivel (192524325)	Code Analyzer (Module 2) + scoring	Python/Java code-quality metrics and software-health scoring
RR Thiyagesh (192572007)	Test Case Generator + Dashboard/Reports + frontend polish (Module 3)	Keyword-triggered test-case templates, dashboard charts and UI styling

Module 1 (Requirements Analysis) was a shared effort between R Kalai Amudhan and Murali Krishnan N; per the Individual Contribution Statement, this report is authored from Murali Krishnan N's individual perspective on that module.

Table 4.8: Budget / Cost

Item	Estimated Cost	Actual Cost
Development tools (VS Code, Git, Python, Flask — free/open-source)	₹0	₹0
Local MySQL / SQLite database	₹0	₹0
Cloud / LLM API usage	₹0 (deliberately avoided, Section 3.4)	₹0

Item	Estimated Cost	Actual Cost
Total	₹0	₹0

CHAPTER 5: CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

This report has documented the design, implementation and testing of Module 1, the Intelligent Requirements Analysis module of SEER (Self-Evolving Engineering Ecosystem for Resilient Software Systems). The module addresses the real engineering problem of inconsistent, time-consuming manual SRS triage by offering a deterministic, fully offline alternative to both plain manual review and non-deterministic LLM-based classification. It reliably segments SRS text in bullet, numbered and paragraph formats, classifies requirements as functional or one of nine non-functional quality-attribute subtypes, audits the document against a 12-item standard checklist derived from ISO/IEC/IEEE 29148, recommends a starting software architecture, and renders a use-case diagram, all validated through manual testing against hand-authored sample SRS documents with encouraging results (~90% classification agreement, correct detection of all deliberately-introduced gaps, and 4/4 correct architecture recommendations). As the entry-point module of the SEER pipeline, its structured output directly enables the Code Analyzer, Test Case Generator and Change Impact Analyzer modules built by the rest of the team, demonstrating a working, integrated application of software-engineering principles across requirements, design, implementation and testing.

5.2 Future Work

- Introduce AI/ML-based (embedding or fine-tuned classifier) requirement classification as an optional mode to improve recall on ambiguous FR/NFR boundary cases identified in Section 4.5, while keeping the current rule engine as a fast, offline default.
- Make the 12-item standard checklist and the architecture-recommendation rules configurable per project domain, rather than fixed at build time.
- Add multi-language SRS support.
- Report a per-classification confidence score so reviewers can quickly identify borderline cases.
- Add GitHub/GitLab integration so an SRS can be pulled directly from a linked repository instead of manual upload.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- Practical experience translating an academic standard (ISO/IEC/IEEE 29148) into an executable checklist that a program can evaluate.
- Deeper understanding of the trade-offs between rule-based and LLM-based NLP approaches for a constrained, gradeable engineering context.
- Hands-on experience with Flask blueprint routing, Jinja2 templating, and SQLAlchemy model relationships in a multi-module application.

Engineering & Professional Skills Developed

- Requirements-engineering discipline — reading and structuring an SRS the way a reviewer or downstream tool would need it.
- Systematic manual test design (Section 4.3) for a module with no ground-truth dataset available.
- Technical writing and evidence-based justification of design decisions (Sections 3.4–3.6), rather than asserting them without support.

Individual Reflection — Murali Krishnan N (192524090)

The most difficult engineering problem I faced was resolving the priority order inside the sentence classifier: an early version of the rules produced several sentences that were both grammatically functional and contained a non-functional keyword, and I had to decide, and defend, which family should win. I traced the disagreement to specific example sentences (Section 4.5), tested both priority orders against the hand-labelled sample SRS documents, and kept the non-functional-first order because it produced fewer missed non-functional requirements overall, even though it introduces the documented "available" false positive.

A key engineering decision I made personally was to scope the missing-requirement checklist to a fixed 12 items drawn directly from ISO/IEC/IEEE 29148, rather than building a configurable checklist through the UI as I initially considered. The evidence behind that decision is in Table 3.6: a configurable checklist would have added meaningful UI and database complexity for a benefit that our own testing (Section 4.4) did not show was necessary for the sample domains we validated against.

The new knowledge I acquired independently was how to design and defend a deterministic NLP-adjacent module without relying on a pre-trained model — reading the ISO/IEC/IEEE 29148 requirement-quality characteristics myself to build the checklist, and working out the word-boundary regex approach in `text_utils.py` after noticing false positives from a naive substring check during my own testing.

If I were to redesign this module, I would add a confidence score to every classification from the start, rather than treating it as future work, since the clearest limitation I found during testing (Section 4.5) was not that the classifier was wrong, but that it gave no signal to the reviewer about which of its decisions were borderline.

REFERENCES

- [1] J. Mucha, A. Kaufmann, and D. Riehle, "A systematic literature review of pre-requirements specification traceability," *Requirements Engineering*, vol. 29, pp. 119–141, 2024.
- [2] J. J. Norheim, E. Rebentisch, D. Xiao, L. Draeger, A. Kerbrat, and O. L. de Weck, "Challenges in applying large language models to requirements engineering tasks," *Design Science*, vol. 10, e16, 2024.
- [3] S. Lubos, A. Felfernig, T. N. T. Tran, D. Garber, M. El Mansi, S. P. Erdeniz, and V.-M. Le, "Leveraging LLMs for the quality assurance of software requirements," in *Proc. IEEE 32nd Int. Requirements Engineering Conf. (RE)*, 2024.
- [4] X. Hou et al., "Large language models for software engineering: A systematic literature review," *ACM Computing Surveys*, 2024.
- [5] B. Wei, "Requirements are all you need: From requirements to code with LLMs," in *Proc. IEEE 32nd Int. Requirements Engineering Conf. (RE)*, 2024, doi: 10.1109/RE59067.2024.00049.
- [6] ISO/IEC/IEEE 29148:2018, *Systems and software engineering — Life cycle processes — Requirements engineering*, International Organization for Standardization, 2018.
- [7] IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*, Institute of Electrical and Electronics Engineers, 1998.
- [8] Flask documentation, Pallets Projects. [Online]. Available: <https://flask.palletsprojects.com>
- [9] SQLAlchemy documentation, SQLAlchemy Project. [Online]. Available: <https://www.sqlalchemy.org>
- [10] python-docx documentation. [Online]. Available: <https://python-docx.readthedocs.io>

APPENDICES

Appendix A – Detailed Calculations

Completeness score formula used by `analyze_requirements()`:

$$\text{completeness_score} = \text{round}(100 \times (\text{count}(\text{functional}) + \text{count}(\text{non_functional})) \div \max(\text{count}(\text{functional}) + \text{count}(\text{non_functional}) + \text{count}(\text{missing}), 1))$$

Worked example (library-management sample SRS): functional = 4, non-functional = 4, missing = 6 → completeness_score = round(100 × 8 ÷ 14) = 57%.

Appendix B – Engineering Drawings / Schematics

Relational schema used to persist Module 1's output (simplified, showing only the fields relevant to this module):

Table	Key Fields	Relationship
Project	id, name, architecture_suggestion, documentation_status	Parent of SRSDocument, Requirement, Diagram
SRSDocument	id, project_id (FK), filename, raw_text, uploaded_at	Belongs to Project
Requirement	id, project_id (FK), text, category, subtype, actor, created_at	Belongs to Project; consumed by TestCase and ChangeAnalysis
Diagram	id, project_id (FK), diagram_type, svg_content, created_at	Belongs to Project (diagram_type = 'use_case' for Module 1)

Appendix C – Source Code / Code Repository Information

Only the core classification excerpt is reproduced here; the complete source is maintained in the project repository (`services/requirement_analyzer.py`, `services/text_utils.py`, `routes/requirements_routes.py`).

```
def _classify_sentence(sentence):
    lower = sentence.lower()
    matched = [f for f, kws in NON_FUNCTIONAL_KEYWORDS.items() if any_keyword_in_text(kws, lower)]
    if matched:
        return {"category": "non_functional", "subtype": matched[0], "actor": ""}
    if re.search(rf"\b{MODAL_VERBS}\b", lower) or lower.startswith(("system", "the system")):
        return {"category": "functional", "subtype": "actor_action", "actor": _extract_actor(sentence)}
    ...
```

Appendix D – Datasheets

Not applicable — Module 1 is a software-only component with no hardware datasheets.

Appendix E – Experimental / Raw Data

Raw JSON output of `analyze_requirements()` for the library-management sample SRS used throughout Chapter 4 (functional and non-functional arrays abbreviated for space; full output verified during testing):

```
{
  "functional": [4 items],
  "non_functional": [4 items],
  "missing": [6 items],
  "actors": ["The Admin", "The Librarian", "The Member", "The system"],
  "architecture": "Three-Tier / MVC Architecture",
  "completeness_score": 57,
  "sentence_count": 8
}
```

Appendix F – Ethics / Institutional Approval

Not applicable — the project involves no human subjects, personal data collection, or clinical/field trial requiring institutional ethics approval.

Appendix G – Project Meeting / Progress Records

See Table 4.6 (Project Milestones) for the recorded weekly progress checkpoints followed by the team.

Appendix H – User / Industry Feedback

Not collected for this capstone iteration — noted as a candidate for future work alongside the items in Section 5.2.

Appendix I – Publications / Patents / Competitions / Product Outcomes

Not applicable at the time of this submission.

Appendix J – Individual Contribution Evidence

See the Individual Contribution Statement and Table 4.7 (Roles & Contribution) for Murali Krishnan N's specific, evidenced contribution to Module 1.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department / faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written / oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2 / SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2 / SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated / system-level engineering	SO1 / SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)